

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Цифровая обработка сигналов и изображений

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

на тему

Алгоритмы цифровой обработки сигналов

Выполнил
студент гр. 250541

В.Ю. Бобрик

Проверил
доцент, к.т.н. каф. ЭВМ

Д.Ю. Перцев

Минск 2026

СОДЕРЖАНИЕ

1 Цель работы.....	3
2 Исходные данные к работе.....	3
3 Теоретические сведения.....	3
4 Выполнение работы.....	5
5 Вывод.....	7

1 Цель работы

Формирование системного представления о математических методах анализа сигналов во временной и частотной областях, а также приобретение практических навыков программной реализации, верификации и сравнительного анализа вычислительной эффективности алгоритмов дискретного преобразования Фурье (ДПФ), быстрого преобразования Фурье (БПФ), свертки и корреляции.

2 Исходные данные к работе

Для выполнения задания необходимо выполнить следующие этапы:

1. Сгенерировать 2 музыкальных файла.
2. Реализовать операцию свертки двух сигналов.
3. Реализовать операцию корреляции двух сигналов;.
4. Для сигналов x и y , а также результата выполнения операции свертки выполнить прямое и обратное преобразование Фурье по методу «БПФ с прореживанием по времени».
5. Построить графики амплитудного и фазового спектра сигналов.
6. Построить графики сигналов во временной области.
7. Экспериментально проверить корректность схем вычисления свертки и корреляции через Фурье-преобразование.
8. Сравнить полученные результаты при разных значениях N (число дискретных отсчетов), оценить эффективность алгоритмов в зависимости от N .

Прямое и обратное преобразования Фурье, вычисление операций свертки и корреляции должны быть представлены в двух вариантах – реализованные студентом самостоятельно и с использованием Open Source библиотек.

3 Теоретические сведения

В реальных системах непрерывный сигнал ($x(t)$) нельзя хранить и обрабатывать напрямую, поэтому его заменяют дискретной последовательностью отсчетов ($x[n]$). Для этого на оси времени вводят шаг дискретизации (T_s), и значения сигнала берутся в моменты времени $t_n = nT_s$, $n = 0, 1, 2, \dots$.

Частота дискретизации $f_s = 1/T_s$ показывает, сколько отсчетов в секунду содержит цифровой сигнал. После дискретизации выполняется квантование — округление значений отсчетов до ближайшего уровня по амплитуде, что позволяет представить каждый отсчет конечным числом бит.

При дискретизации важна связь между частотой дискретизации f_s и максимальной значимой частотой $f_{\max}$ исходного сигнала. Для корректного

восстановления сигнала необходимо, чтобы частота дискретизации была не менее чем вдвое больше максимальной частоты спектра: $f_s \geq f_{\max}$.

Если это условие нарушается, возникает эффект наложения спектров (aliasing): высокочастотные компоненты проецируются в низкочастотную область и искажают спектр дискретизированного сигнала. На графиках пособия показано, как при недостаточной частоте дискретизации разные непрерывные сигналы могут давать одинаковые дискретные последовательности.

Для хранения звука используется, как правило, формат WAV, в котором сигнал представлен последовательностью целочисленных отсчётов с заданной частотой дискретизации и разрядностью. Файлы с изображениями (JPEG) и звуком (MP3, WAV) по сути содержат дискретизированные данные, к которым можно применять алгоритмы цифровой обработки.

Частота дискретизации аудио (например, 8 кГц, 16 кГц, 44,1 кГц) определяет частотный диапазон, который может быть представлен без aliasing. Разрядность (8, 16 бит и т.д.) задаёт число уровней квантования амплитуды и влияет на шум квантования и динамический диапазон.

Дискретное преобразование Фурье – основной инструмент анализа спектра дискретного сигнала. ДПФ позволяет представить последовательность $x[n]$ длины N как сумму комплексных гармоник с различными частотами и фазами.

Прямое ДПФ определяется суммой

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j 2\pi kn/N}, k = 0, \dots, N-1,$$

а обратное преобразование позволяет восстановить $x[n]$ по спектру $X[k]$. Индекс k связан с физической частотой $f_k = kf_s/N$, что важно для интерпретации результатов спектрального анализа аудиосигналов.

Прямое вычисление ДПФ по формуле требует порядка N^2 комплексных операций, что становится проблемой при больших N . Быстрое преобразование Фурье (БПФ) позволяет сократить число операций до $O(N \log_2 N)$ за счёт разложения ДПФ на серию более простых подзадач.

Среди алгоритмов БПФ отдельно выделяется вариант с прореживанием по времени (Decimation in Time). Его идея состоит в разбиении исходной последовательности $x[n]$ на две подпоследовательности: отсчёты с чётными индексами и отсчёты с нечётными индексами, после чего для каждой подпоследовательности вычисляется ДПФ меньшего размера.

4 Выполнение работы

Для выполнения работы использовался Python 3.14 с плагином Jupyter Notebook. Исходный код ноутбука и результаты интерпретации представлены ниже.

```
#### 0. Инициализация
import sys
sys.path.append('../src')

import numpy as np
import matplotlib.pyplot as plt

from signals import generate_signal, read_signal
from fft_time_decimation import fft_dit, ifft_dit
from convolution import conv_time, conv_fft
from correlation import corr_time, corr_fft

plt.rcParams['figure.figsize'] = (10, 4)
plt.rcParams['grid.alpha'] = 0.3

#### 1a. Генерация сигналов x и y
# параметры
fs = 400          # частота дискретизации, Гц
duration = 3.0    # длительность, с

def sigX_func(t: np.ndarray) -> np.ndarray:
    return 2 * np.sin(3 * t) + np.cos(2 * t + 1)

generate_signal('sigX.wav', sigX_func, fs=fs, duration=duration)
print("Файл sigX.wav сгенерирован.")

def sigY_func(t: np.ndarray) -> np.ndarray:
    return 1.5 * np.sin(5 * t + 0.5) + 0.5 * np.cos(4 * t)

generate_signal('sigY.wav', sigY_func, fs=fs, duration=duration)
print("Файл sigY.wav сгенерирован.")

#### 1b. Чтение сигналов
fs1, x = read_signal('sigX.wav')
fs2, y = read_signal('sigY.wav')

t = np.arange(len(x)) / fs1
max_time = 3
mask = t < max_time

plt.plot(t[mask], x[mask], label='x(t) из sigX.wav')
plt.plot(t[mask], y[mask], label='y(t) из sigY.wav', alpha=0.7)
plt.xlabel('t, c')
```

```
plt.ylabel('Амплитуда')
plt.title('Фрагменты сгенерированных сигналов во временной области')
plt.grid(True)
plt.legend()
plt.show()
```

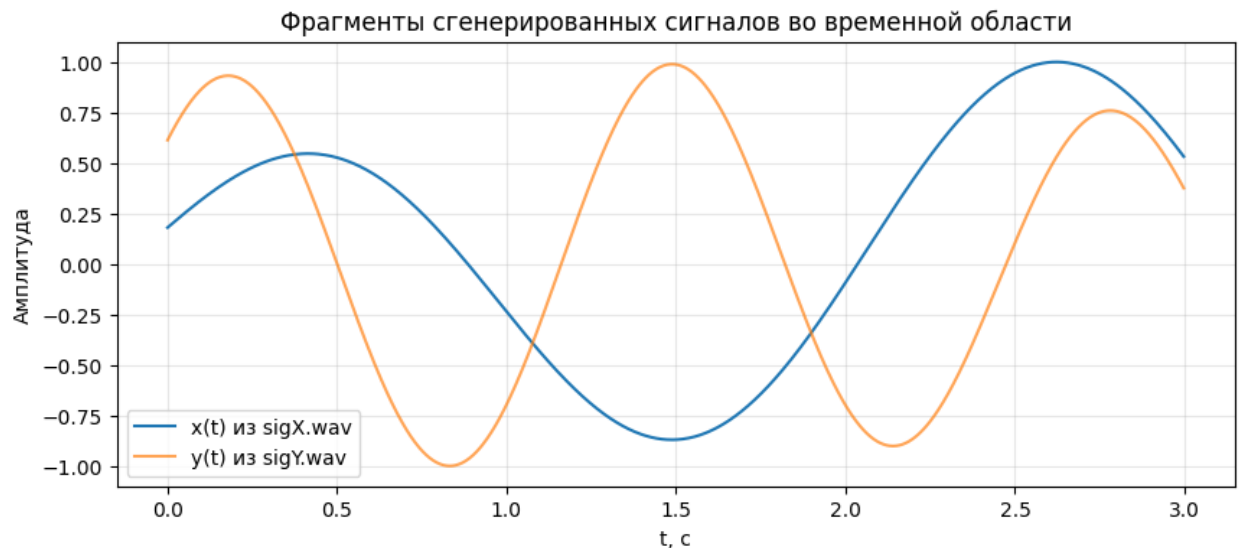


Рисунок 1 – Сгенерированные сигналы

```
#### 2а. Свёртка двух сигналов во временной области
# подготовка сигналов для свертки
x_list = [complex(val) for val in x]
y_list = [complex(val) for val in y]

len_x = len(x_list)
len_y = len(y_list)

# 1. Графики x(t) и y(t) на всём интервале
t = np.arange(len(x)) / fs1

plt.figure(figsize=(10, 4))
plt.plot(t, x, label='x(t) из sigX.wav')
plt.plot(t, y, label='y(t) из sigY.wav', alpha=0.7)
plt.xlabel('t, c')
plt.ylabel('Амплитуда')
plt.title('Сигналы x(t) и y(t) во временной области')
plt.grid(True)
plt.legend()
plt.show()

# 2. Свёртка

# подготовка списков комплексных значений для conv_time
x_list = [complex(val) for val in x]
y_list = [complex(val) for val in y]
```

```

# свёртка (моя)
s_time = conv_time(x_list, y_list)
s_time_np = np.array(s_time)

# свёртка через NumPy
s_np = np.convolve(x, y, mode='full')

L = len(s_time_np)
t_conv = np.arange(L) / fs1

# 3. Сравнение результатов свёртки на одном графике
plt.figure(figsize=(10, 4))
plt.plot(t_conv, s_time_np.real, label='свёртка моя (conv_time)')
plt.plot(t_conv, s_np, '--', label='свёртка NumPy (np.convolve)', alpha=0.7)
plt.xlabel('t, c')
plt.ylabel('s(t)')
plt.title('Сравнение свёртки: собственная реализация и NumPy')
plt.grid(True)
plt.legend()
plt.show()

print("Максимальное расхождение:",
      np.max(np.abs(s_time_np.real - s_np)))

```

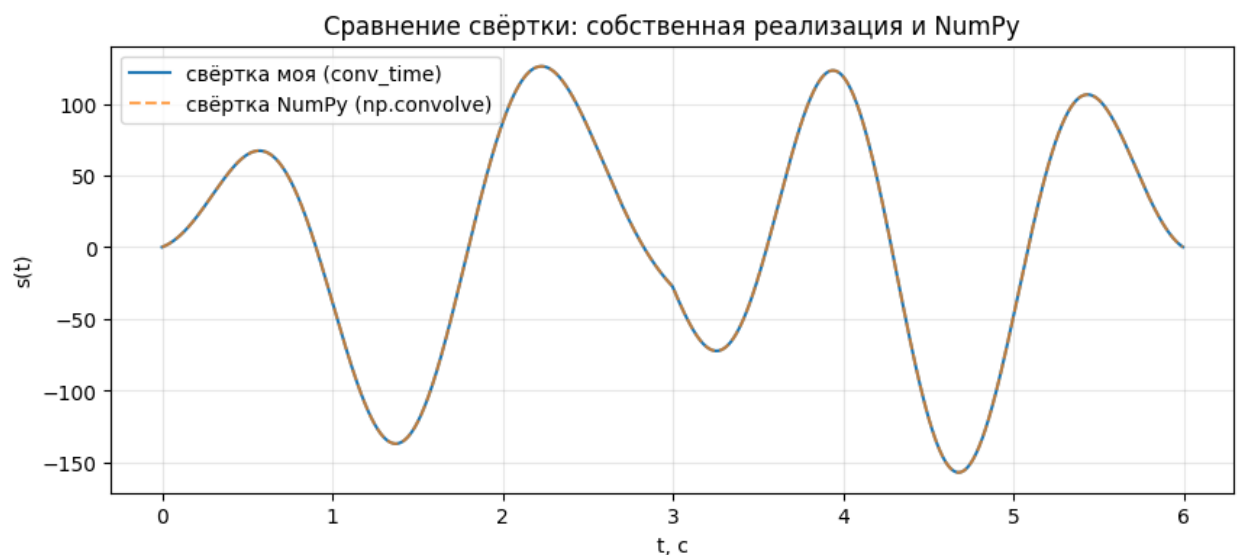


Рисунок 2 – Свёртка сигналов

```

#### 2б. Корреляция двух сигналов во временной области
# 1. Сигналы  $x(t)$  и  $y(t)$ 
t = np.arange(len(x)) / fs1

plt.figure(figsize=(10, 4))

```

```

plt.plot(t, x, label='x(t) из sigX.wav')
plt.plot(t, y, label='y(t) из sigY.wav', alpha=0.7)
plt.xlabel('t, с')
plt.ylabel('Амплитуда')
plt.title('Сигналы x(t) и y(t) во временной области')
plt.grid(True)
plt.legend()
plt.show()

# 2. Корреляция моя

x_list = [complex(val) for val in x]
y_list = [complex(val) for val in y]

r_xy_time = corr_time(x_list, y_list) # моя
r_xy_time_np = np.array(r_xy_time)

N = len(x)
m_values = np.arange(-(N-1), N)

# 3. Корреляция через NumPy

r_xy_np = np.correlate(x, y, mode='full')

# 4. Сравнение результатов

plt.figure(figsize=(10, 4))
plt.plot(m_values, r_xy_time_np.real, label='corr_time (временная область)')
plt.plot(m_values, r_xy_np, '--', label='numpy.correlate (full)', alpha=0.7)
plt.xlabel('лаг m')
plt.ylabel('R_xy[m]')
plt.title('Сравнение корреляции: собственная реализация и NumPy')
plt.grid(True)
plt.legend()
plt.show()

print("Максимальное расхождение:",
      np.max(np.abs(r_xy_time_np.real - r_xy_np)))

```




Рисунок 3 – Корреляция сигналов

```
#### 2с. FFT & IFFT
# 1. Подготовка трёх сигналов: x, y и s
s = s_time_np.real

# чтобы использовать рекурсивный БПФ, длины должны быть степенью двойки
def pad_to_pow2(z: np.ndarray):
    N = len(z)
    N_fft = 1
    while N_fft < N:
        N_fft *= 2
    z_pad = np.zeros(N_fft, dtype=complex)
    z_pad[:N] = z.astype(complex)
    return z_pad, N, N_fft

x_pad, Nx, Nx_fft = pad_to_pow2(x)
y_pad, Ny, Ny_fft = pad_to_pow2(y)
s_pad, Ns, Ns_fft = pad_to_pow2(s)

print("Длины после дополнения до степени двойки:",
      "x:", Nx_fft, "y:", Ny_fft, "s:", Ns_fft)

# 2. Прямое БПФ: моя реализация и NumPy
X_fft = fft_dit(list(x_pad))
Y_fft = fft_dit(list(y_pad))
S_fft = fft_dit(list(s_pad))

X_np = np.fft.fft(x_pad)
Y_np = np.fft.fft(y_pad)
S_np = np.fft.fft(s_pad)

# 3. Один график с исходными сигналами во времени
```

```

t_x = np.arange(Nx) / fs1
t_y = np.arange(Ny) / fs1
t_s = np.arange(Ns) / fs1

plt.figure(figsize=(10, 6))
plt.subplot(3, 1, 1)
plt.plot(t_x, x, label='x(t)')
plt.ylabel('Амплитуда')
plt.title('Исходные сигналы во временной области')
plt.grid(True)

plt.subplot(3, 1, 2)
plt.plot(t_y, y, label='y(t)', color='C1')
plt.ylabel('Амплитуда')
plt.grid(True)

plt.subplot(3, 1, 3)
plt.plot(t_s, s, label='s(t) = x*y', color='C2')
plt.xlabel('t, c')
plt.ylabel('Амплитуда')
plt.grid(True)

plt.tight_layout()
plt.show()

# 4. Сравнение своей реализации и NumPy для каждого сигнала
freq_x = np.fft.fftfreq(Nx_fft, d=1/fs1)
freq_y = np.fft.fftfreq(Ny_fft, d=1/fs1)
freq_s = np.fft.fftfreq(Ns_fft, d=1/fs1)

X_fft_np = np.array(X_fft)
Y_fft_np = np.array(Y_fft)
S_fft_np = np.array(S_fft)

print("Макс. расхождение спектров X:", np.max(np.abs(X_fft_np - X_np)))
print("Макс. расхождение спектров Y:", np.max(np.abs(Y_fft_np - Y_np)))
print("Макс. расхождение спектров S:", np.max(np.abs(S_fft_np - S_np)))

# 5. Обратное БПФ (моя)
x_rec_full = np.array(ifft_dit(list(X_fft)))
y_rec_full = np.array(ifft_dit(list(Y_fft)))
s_rec_full = np.array(ifft_dit(list(S_fft)))

# 6. Обратное БПФ через NumPy
x_ifft_np_full = np.fft.ifft(X_np)
y_ifft_np_full = np.fft.ifft(Y_np)
s_ifft_np_full = np.fft.ifft(S_np)

# 7. Берём только первые длины исходных сигналов

```

```

Nx = len(x)
Ny = len(y)
Ns = len(s)

x_rec = x_rec_full[:Nx]
y_rec = y_rec_full[:Ny]
s_rec = s_rec_full[:Ns]

x_ifft_np = x_ifft_np_full[:Nx]
y_ifft_np = y_ifft_np_full[:Ny]
s_ifft_np = s_ifft_np_full[:Ns]

plt.figure(figsize=(10, 6))

plt.subplot(3, 1, 1)
plt.plot(t_x, x, label='x(t) исходный')
plt.plot(t_x, x_rec.real, '--', label='x(t) ifft_dit', alpha=0.7)
plt.plot(t_x, x_ifft_np.real, ':', label='x(t) numpy.ifft', alpha=0.7)
plt.ylabel('Амплитуда')
plt.title('Восстановление сигналов после обратного БПФ')
plt.grid(True)
plt.legend()

plt.subplot(3, 1, 2)
plt.plot(t_y, y, label='y(t) исходный', color='C1')
plt.plot(t_y, y_rec.real, '--', label='y(t) ifft_dit', alpha=0.7, color='C3')
plt.plot(t_y, y_ifft_np.real, ':', label='y(t) numpy.ifft', alpha=0.7,
color='C4')
plt.ylabel('Амплитуда')
plt.grid(True)
plt.legend()

plt.subplot(3, 1, 3)
plt.plot(t_s, s, label='s(t) исходный', color='C2')
plt.plot(t_s, s_rec.real, '--', label='s(t) ifft_dit', alpha=0.7, color='C5')
plt.plot(t_s, s_ifft_np.real, ':', label='s(t) numpy.ifft', alpha=0.7,
color='C6')
plt.xlabel('t, c')
plt.ylabel('Амплитуда')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

print("Макс. расхождение x vs ifft_dit:", np.max(np.abs(x - x_rec.real)))
print("Макс. расхождение x vs numpy.ifft:", np.max(np.abs(x - x_ifft_np.real)))
print("Макс. расхождение y vs ifft_dit:", np.max(np.abs(y - y_rec.real)))
print("Макс. расхождение y vs numpy.ifft:", np.max(np.abs(y - y_ifft_np.real)))

```

```
print("Макс. расхождение s vs ifft_dit:", np.max(np.abs(s - s_rec.real)))
print("Макс. расхождение s vs numpy.ifft:", np.max(np.abs(s - s_ifft_np.real)))
```

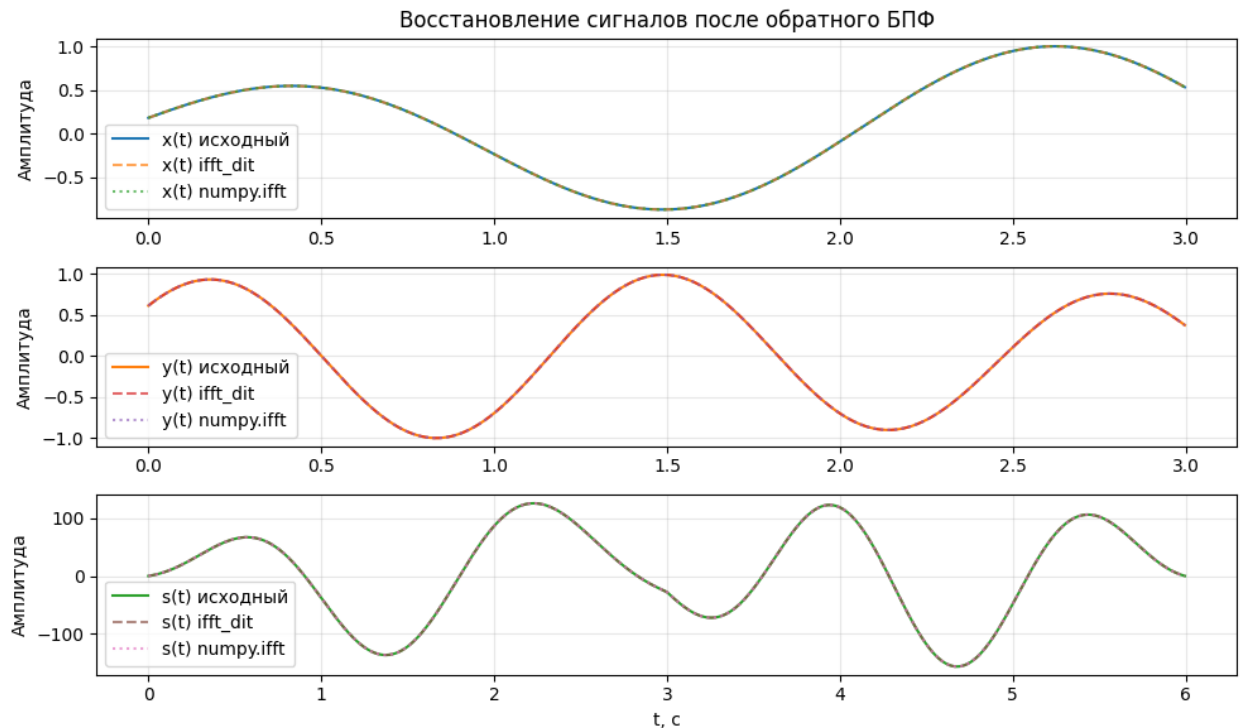


Рисунок 4 – Результат прямого и обратного БПФ

2d. Амплитудные и фазовые спектры сигналов x, y и s

```
X_fft_np = np.array(X_fft)
Y_fft_np = np.array(Y_fft)
S_fft_np = np.array(S_fft)

Amp_X = np.abs(X_fft_np)
Amp_Y = np.abs(Y_fft_np)
Amp_S = np.abs(S_fft_np)

Phase_X = np.angle(X_fft_np)
Phase_Y = np.angle(Y_fft_np)
Phase_S = np.angle(S_fft_np)

plt.figure(figsize=(10, 8))

# Амплитудные спектры
plt.subplot(3, 2, 1)
#plt.plot(freq_x, Amp_X)
plt.stem(freq_x, Amp_X)
plt.xlim(-2, 2)
plt.ylabel('Амплитуда')
plt.title('Амплитудный спектр |X(f)|')
plt.grid(True)
```

```

plt.subplot(3, 2, 3)
#plt.plot(freq_y, Amp_Y, color='C1')
plt.stem(freq_y, Amp_Y)
plt.xlim(-2, 2)
plt.ylabel('Амплитуда')
plt.title('Амплитудный спектр  $|Y(f)|$ ')
plt.grid(True)

plt.subplot(3, 2, 5)
#plt.plot(freq_s, Amp_S, color='C2')
plt.stem(freq_s, Amp_S)
plt.xlim(-2, 2)
plt.xlabel('f, Гц')
plt.ylabel('Амплитуда')
plt.title('Амплитудный спектр  $|S(f)|$ ')
plt.grid(True)

# Фазовые спектры
plt.subplot(3, 2, 2)
#plt.plot(freq_x, Phase_X)
plt.stem(freq_x, Phase_X)
plt.xlim(-2, 2)
plt.ylabel('Фаза, рад')
plt.title('Фазовый спектр  $\arg X(f)$ ')
plt.grid(True)

plt.subplot(3, 2, 4)
plt.stem(freq_y, Phase_Y)
plt.xlim(-2, 2)
plt.ylabel('Фаза, рад')
plt.title('Фазовый спектр  $\arg Y(f)$ ')
plt.grid(True)

plt.subplot(3, 2, 6)
plt.stem(freq_s, Phase_S)
plt.xlim(-2, 2)
plt.xlabel('f, Гц')
plt.ylabel('Фаза, рад')
plt.title('Фазовый спектр  $\arg S(f)$ ')
plt.grid(True)

plt.tight_layout()
plt.show()

```

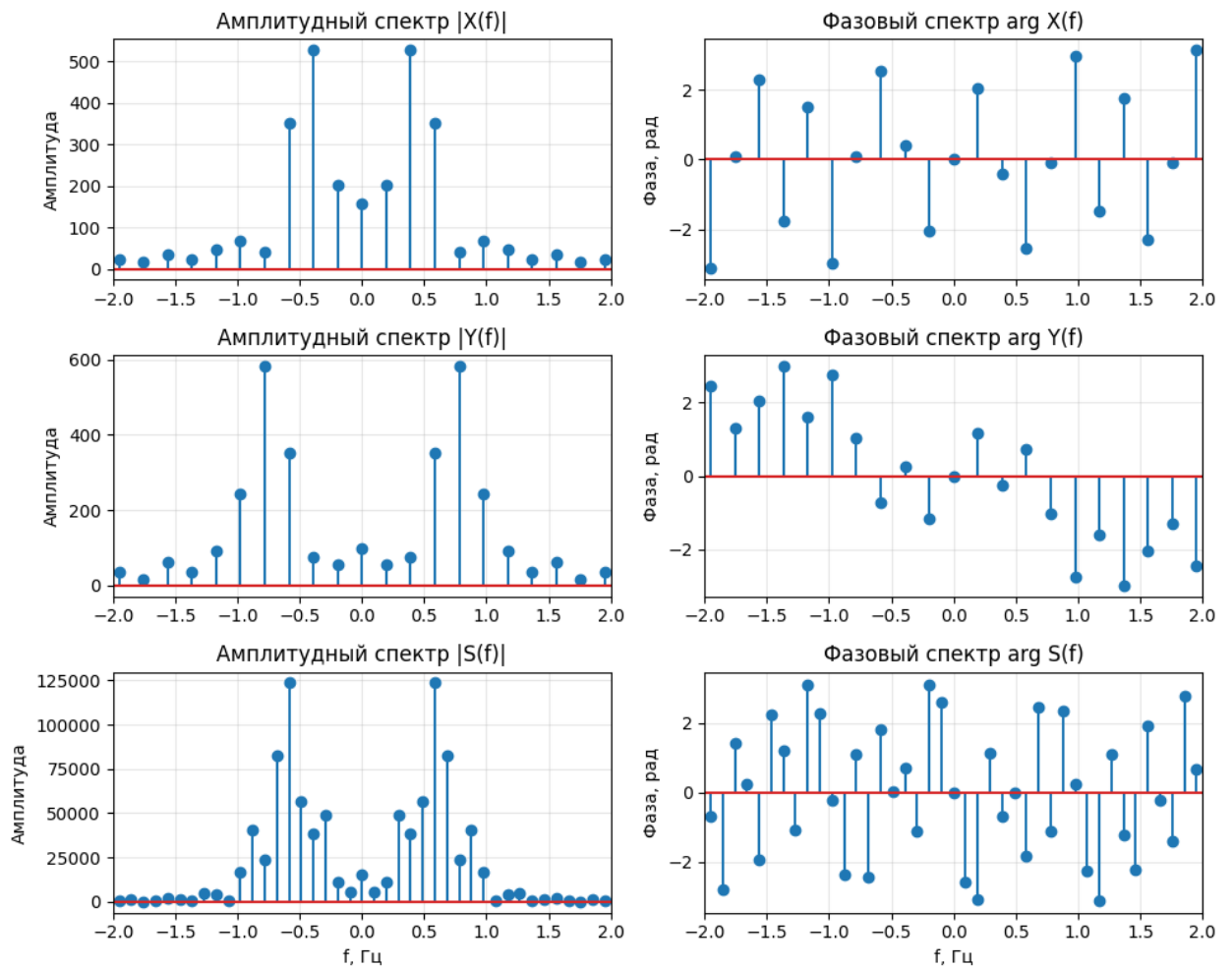


Рисунок 5 – Амплитудные и фазовые спектры сигналов

2е. Графики сигналов во временной области

```
s = s_time_np.real
t_x = np.arange(len(x)) / fs1
t_y = np.arange(len(y)) / fs1
t_s = np.arange(len(s)) / fs1

plt.figure(figsize=(10, 6))
plt.subplot(3, 1, 1)
plt.plot(t_x, x)
plt.ylabel('Амплитуда')
plt.title('Сигнал x(t) во временной области')
plt.grid(True)

plt.subplot(3, 1, 2)
plt.plot(t_y, y, color='C1')
plt.ylabel('Амплитуда')
plt.title('Сигнал y(t) во временной области')
plt.grid(True)

plt.subplot(3, 1, 3)
```

```
plt.plot(t_s, s, color='C2')
plt.xlabel('t, c')
plt.ylabel('Амплитуда')
plt.title('Сигнал s(t) = x*y во временной области')
plt.grid(True)

plt.tight_layout()
plt.show()
```

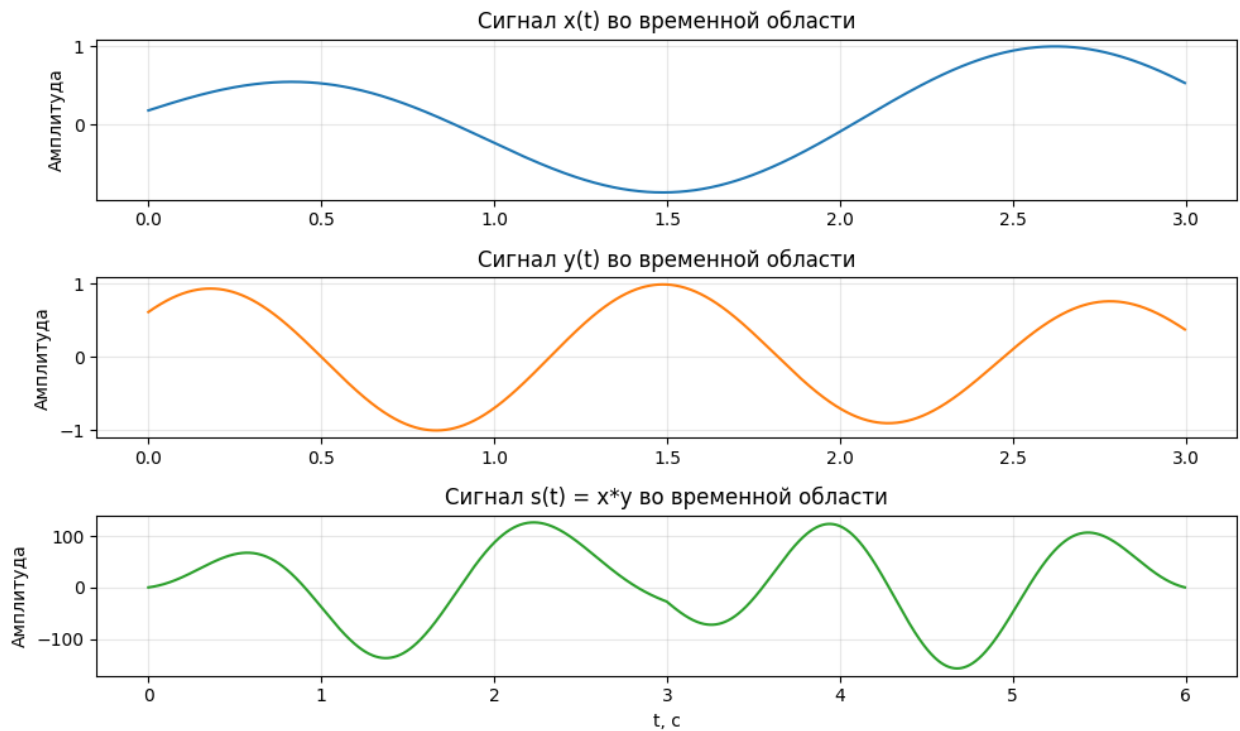


Рисунок 6 – Сигналы во временной области

```
#### 2f. Проверка свёртки и корреляции через Фурье-преобразование
# подготовка списков complex
x_list = [complex(val) for val in x]
y_list = [complex(val) for val in y]

# 1. Свёртка во временной области (моя)
s_time = np.array(conv_time(x_list, y_list))

# 2. Свёртка через БПФ (моя)
s_fft = np.array(conv_fft(x_list, y_list))

# 3. Свёртка через NumPy (эталон)
s_np = np.convolve(x, y, mode='full')

# 4. Сравнение по максимуму расхождения
print("Свёртка:")
print("max |s_time - s_fft| =", np.max(np.abs(s_time.real - s_fft.real)))
print("max |s_time - s_np| =", np.max(np.abs(s_time.real - s_np)))
```

```

print("max |s_fft - s_np | =", np.max(np.abs(s_fft.real - s_np)))

# 5. Графическое сравнение (фрагмент)
L = len(s_np)
t_conv = np.arange(L) / fs1

max_time_conv = 3 # показать первые 3 сек результата
mask = t_conv < max_time_conv

plt.figure(figsize=(10, 4))
plt.plot(t_conv[mask], s_time.real[mask], label='conv_time (моя)')
plt.plot(t_conv[mask], s_fft.real[mask], '--', label='conv_fft (БПФ)', alpha=0.7)
plt.plot(t_conv[mask], s_np[mask], ':', label='numpy.convolve', alpha=0.7)
plt.xlabel('t, c')
plt.ylabel('Амплитуда')
plt.title('Сравнение свёртки: временная область, БПФ и NumPy')
plt.grid(True)
plt.legend()
plt.show()

```

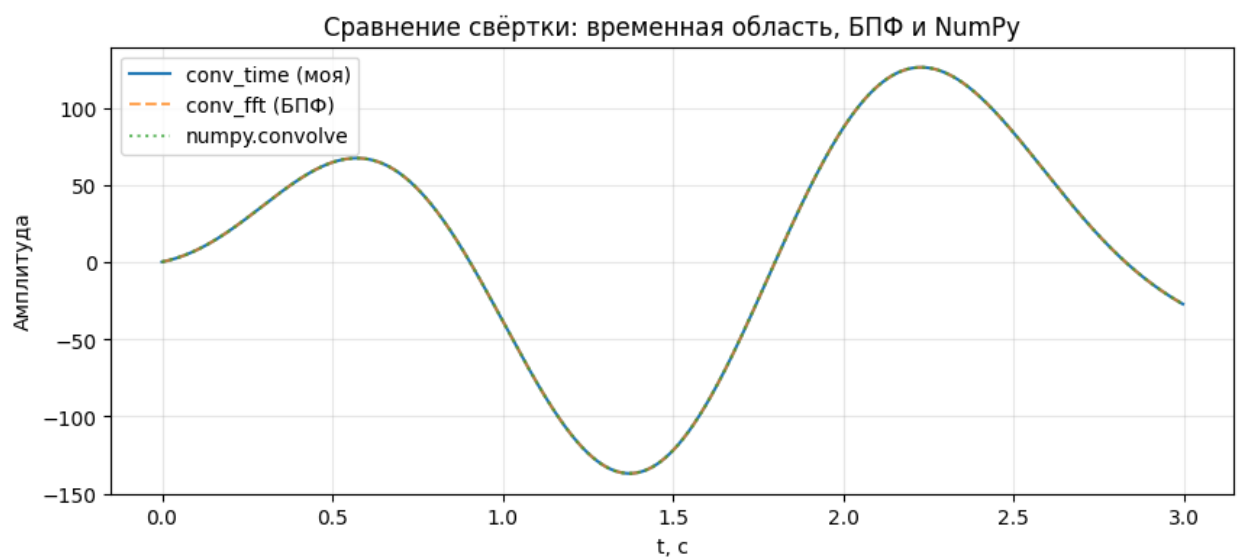


Рисунок 7 – Свертка сигналов через БПФ

```

# 1. Корреляция во временной области (моя)
r_time = np.array(corr_time(x_list, y_list))

# 2. Корреляция через БПФ (моя)
r_fft = np.array(corr_fft(x_list, y_list))

# 3. Корреляция через NumPy (эталон)

r_np = np.correlate(x, y, mode='full')

N = len(x)

```



```

m_values = np.arange(-(N - 1), N)

print("Корреляция:")
print("max |r_time - r_fft| =", np.max(np.abs(r_time.real - r_fft.real)))
print("max |r_time - r_np| =", np.max(np.abs(r_time.real - r_np)))
print("max |r_fft - r_np| =", np.max(np.abs(r_fft.real - r_np)))

# Графическое сравнение
plt.figure(figsize=(10, 4))
plt.plot(m_values, r_time.real, label='corr_time (время)')
plt.plot(m_values, r_fft.real, '--', label='corr_fft (БПФ)', alpha=0.7)
plt.plot(m_values, r_np, ':', label='numpy.correlate', alpha=0.7)
plt.xlabel('лаг m')
plt.ylabel('R_xy[m]')
plt.title('Сравнение корреляции: временная область, БПФ и NumPy')
plt.grid(True)
plt.legend()
plt.show()

```

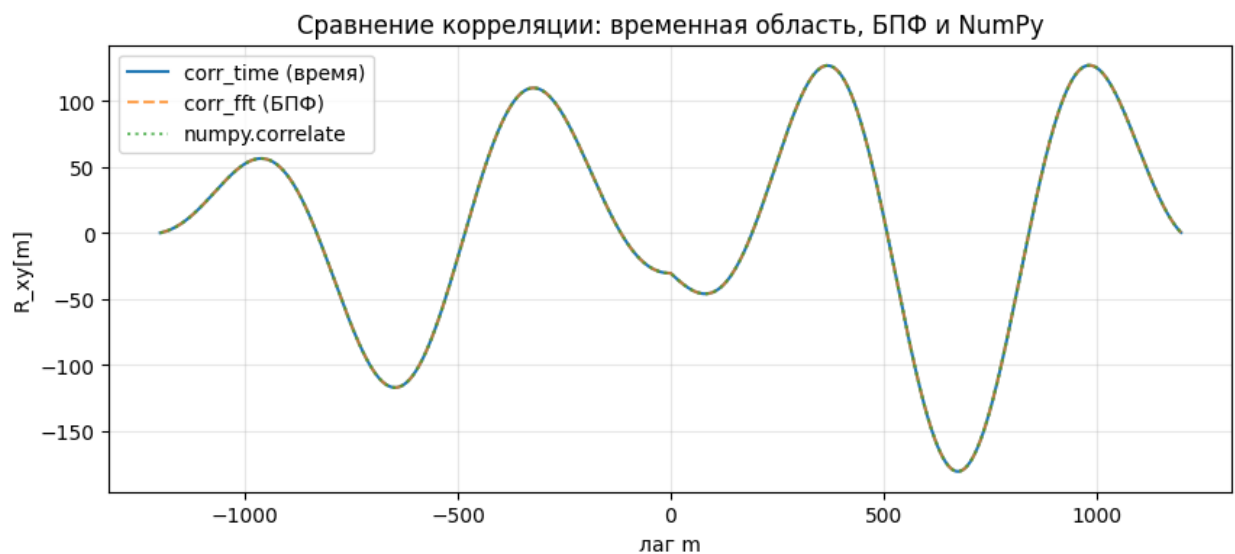


Рисунок 8 – Корреляция сигналов через БПФ

3. Влияние числа отсчётов N на эффективность алгоритмов

```

import time

Ns = [256, 512, 1024, 2048, 4096]
results_conv = []
results_corr = []

for N in Ns:
    # случайные тестовые сигналы
    xN = np.random.randn(N)
    yN = np.random.randn(N)

```

```

x_list = [complex(val) for val in xN]
y_list = [complex(val) for val in yN]

# --- свёртка ---
t0 = time.perf_counter()
s_time = conv_time(x_list, y_list)
t1 = time.perf_counter()

t2 = time.perf_counter()
s_fft = conv_fft(x_list, y_list)
t3 = time.perf_counter()

t4 = time.perf_counter()
s_np = np.convolve(xN, yN, mode='full')
t5 = time.perf_counter()

results_conv.append([
    N,
    t1 - t0,    # conv_time
    t3 - t2,    # conv_fft
    t5 - t4     # numpy.convolve
])

# --- корреляция ---
t0 = time.perf_counter()
r_time = corr_time(x_list, y_list)
t1 = time.perf_counter()

t2 = time.perf_counter()
r_fft = corr_fft(x_list, y_list)
t3 = time.perf_counter()

t4 = time.perf_counter()
r_np = np.correlate(xN, yN, mode='full')
t5 = time.perf_counter()

results_corr.append([
    N,
    t1 - t0,    # corr_time
    t3 - t2,    # corr_fft
    t5 - t4     # numpy.correlate
])

results_conv = np.array(results_conv)
results_corr = np.array(results_corr)

print("Свёртка (время в секундах):")
print("N\tconv_time\tconv_fft\tnp.convolve")
for row in results_conv:

```

```

print(f"{int(row[0])}\t{row[1]:.6f}\t{row[2]:.6f}\t{row[3]:.6f}")

print("\nКорреляция (время в секундах):")
print("N\tcorr_time\tcorr_fft\tnp.correlate")
for row in results_corr:
    print(f"{int(row[0])}\t{row[1]:.6f}\t{row[2]:.6f}\t{row[3]:.6f}")

...

Свёртка (время в секундах):
N      conv_time    conv_fft    np.convolve
256    0.090312      0.009755    0.000132
512    0.291748      0.007698    0.000087
1024    1.500774      0.074326    0.000493
2048    6.280593      0.069921    0.000974
4096    23.987050     0.099650    0.004084

Корреляция (время в секундах):
N      corr_time    corr_fft    np.correlate
256    0.031708      0.007548    0.000067
512    0.071030      0.012391    0.000087
1024    0.908108      0.063877    0.000451
2048    1.686624      0.052026    0.000511
4096    6.370807      0.160518    0.005577
...

```

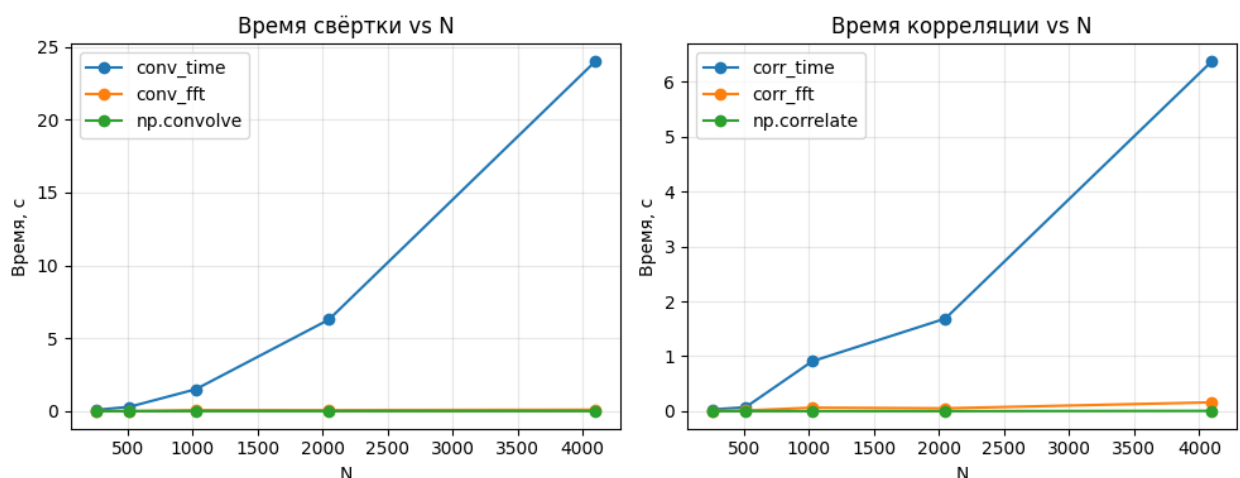


Рисунок 9 – Графики зависимости сложности алгоритмов от количества отсчетов

5 Вывод

Сформировал представление о математических методах анализа сигналов во временной и частотной областях, а также приобрел практические навыки программной реализации и сравнительного анализа вычислительной эффективности алгоритмов дискретного преобразования Фурье (ДПФ), быстрого преобразования Фурье (БПФ), свертки и корреляции.